

Lección 6: Safe Refactoring

Refactorización Segura con TDD

Agenda

- ¿Qué es Safe Refactoring?
 - Red → Green → Refactor Cycle
 - Patrones de Refactoring Aplicados
 - Técnicas de Seguridad
 - Métricas y ROI
-

¿Qué es Safe Refactoring?

“Refactoring es reestructurar código sin cambiar su comportamiento externo” -
Martin Fowler

Principios:

-  **Preserve Behavior** (preservar comportamiento): Funcionalidad idéntica
 -  **Test-Driven**: Tests como red de seguridad (safety net)
 -  **Improve Structure** (mejorar estructura): Mejor diseño interno
 -  **Small Steps** (pasos pequeños): Cambios incrementales
-

La Regla de Oro

Red → Green → Refactor Cycle:

//  *RED: Test falla*

```
it('should calculate bulk discount', () => {
  expect(calculateBulkDiscount(item, 5)).toBe(14.995)
})

// 🟢 GREEN: Hacer pasar (rápido y sucio)
function calculateBulkDiscount() {
  return 14.995
}

// ♻️ REFACTOR: Mejorar calidad (tests verdes)
function calculateBulkDiscount(item, quantity) {
  if (quantity >= businessRules.bulkDiscount.threshold) {
    return roundToCents(item.price * quantity * 0.1)
  }
  return 0
}
```

Test Safety Net

```
// Tests pasan ANTES del refactoring
describe('Before Refactor', () => {
  it('calculates bulk discount', () => {
    expect(calculateBulkDiscount(mockItem, 5)).toBe(14.995) // ✅
  })
  it('no discount for < 5 items', () => {
    expect(calculateBulkDiscount(mockItem, 3)).toBe(0) // ✅
  })
})

// MISMOS tests pasan DESPUÉS del refactoring
describe('After Refactor', () => {
  it('calculates bulk discount', () => {
    expect(calculateBulkDiscount(mockItem, 5)).toBe(14.995) // ✅
  })
  it('no discount for < 5 items', () => {
    expect(calculateBulkDiscount(mockItem, 3)).toBe(0) // ✅
  })
})
```

Patrón 1: Extract Constants

Paso a paso:

```
// 1. Tests verdes antes   
expect(calculateBulkDiscount(item, 5)).toBe(14.995)  
  
// 2. Extraer constante  
const BULK_THRESHOLD = 5  
  
// 3. Verificar tests   
expect(calculateBulkDiscount(item, 5)).toBe(14.995)  
  
// 4. Centralizar  
export const businessRules = {  
  bulkDiscount: { threshold: 5, percentage: 0.1 },  
}  
  
// 5. Verificar tests 
```

Pequeños pasos, siempre verde

Patrón 2: Replace Primitive Obsession

Antes:

```
// Formateo disperso  
const price = new Intl.NumberFormat('en-US', {  
  style: 'currency',  
  currency: 'USD',  
}).format(product.price)
```

Después:

```
// Custom hook  
const { formatPriceWithCurrency } = useCurrencyFormat()  
<span>{formatPriceWithCurrency(product.price)}</span>
```

Tests garantizan mismo resultado

Técnica: Mikado Method

Refactoring grande en pasos pequeños:

```
const refactoringPlan = {
  goal: 'Centralizar reglas de negocio',
  steps: [
    {
      name: 'Extraer números mágicos',
      safety: 'Todos los tests de cálculo pasan',
      rollback: 'Revertir a literales',
    },
    {
      name: 'Crear hook de validación',
      safety: 'Todos los tests de formularios pasan',
      rollback: 'Restaurar validación inline',
    },
  ],
}
```

Cada paso es atómico y reversible

Técnica: Parallel Change

Branch by Abstraction:

```
// 1. Crear interfaz
interface DiscountCalculator {
  calculate(item, quantity): number
}

// 2. Old implementation (mantener)
class LegacyCalculator implements DiscountCalculator

// 3. New implementation (desarrollar)
class ModernCalculator implements DiscountCalculator

// 4. Feature flag para cambiar
const calculator = useModern() ? modern : legacy

// 5. Cuando funciona, eliminar legacy :)
```

Técnica: Approval Testing

Capturar comportamiento actual antes de refactorizar:

```
describe('Approval Tests', () => {
  it('maintains exact calculations', () => {
    const testCases = [
      { item: { price: 29.99 }, qty: 1, expected: 0 },
      { item: { price: 29.99 }, qty: 5, expected: 14.995 },
      { item: { price: 29.99 }, qty: 10, expected: 29.99 },
    ]

    // Después del refactoring, mismos inputs = mismos outputs
    testCases.forEach(({ item, qty, expected }) => {
      expect(calculate(item, qty)).toBe(expected)
    })
  })
})
```

Métricas de Calidad: ¿Qué Medir? (1/2)

1. Cyclomatic Complexity

```
// Complejidad = # de caminos independientes
if (qty >= 5) {
  // +1 camino
  if (price > 100) {
    // +1 camino
  }
}
// Ideal: < 5, Aceptable: < 10, Refactorizar: > 10
```

2. Lines of Code (LoC): Tamaño total en cantidad de líneas de código

3. Duplication: % de código repetido

Métricas de Calidad: ¿Qué Medir? (2/2)

4. Test Coverage: % de código ejecutado por tests

- Objetivo: > 80% (funciones críticas: 100%)

5. Bug Reports: Cantidad de Bugs reportados mensualmente

6. Maintainability Index: Score 0-100

- > 80 = Fácil de mantener
 - < 60 = Difícil de mantener
-

Métricas: Antes vs Después

Antes:

- Cyclomatic Complexity: 15
- Lines of Code: 2847
- Duplication: 23%
- Test Coverage: 78%
- Bug Reports: 12/month

Después:

- Cyclomatic Complexity: 8 (-47%)
 - Lines of Code: 2234 (-22%)
 - Duplication: 5% (-78%)
 - Test Coverage: 94% (+16%)
 - Bug Reports: 3/month (-75%)
-

Estrategia: Boy Scout Rule

“Deja el campamento más limpio de como lo encontraste”

```
//  Antes: Código sucio - ¿Qué está mal?  
function calc(items) {  
  let t=0;for(let i=0;i<items.length;i++){...}  
}
```

```
// Problemas:  
// 1. Nombre vago: "calc" - ¿calcular qué?  
// 2. Variable críptica: "t" - ¿qué representa?  
// 3. Sin tipos: ¿qué es items? ¿qué retorna?  
// 4. For loop manual: difícil de leer  
// 5. Sin separación: todo en 1 línea ilegible
```

Boy Scout Regla: Limpiar Mientras Trabajas

```
// ✅ Después: Limpiado mientras añadías feature  
function calculateTotal(items: CartItem[]): number {  
  return items.reduce((total, item) => {  
    const subtotal = item.price * item.quantity  
    const discount = calculateBulkDiscount(item, item.quantity)  
    return total + subtotal - discount  
  }, 0)  
}
```

```
// Mejoras:  
// ✅ Nombre descriptivo: calculateTotal  
// ✅ Tipos explícitos: CartItem[], number  
// ✅ Functional approach: reduce en vez de for  
// ✅ Variables con sentido: subtotal, discount  
// ✅ Legibilidad: cada paso en su línea
```

Estrategia: Strangler Fig Pattern

Reemplazar gradualmente sistema viejo:

```
class CartService {  
  // Legacy (siendo reemplazado)  
  calculateTotalLegacy(items): number {  
    /* old */  
  }  
  
  // New (estrangulando al viejo)  
  calculateTotal(items: CartItem[]): number {  
    /* clean */  
  }  
}
```

```
// Router gradual
getTotal(items: CartItem[]): number {
  if (this.useNewImplementation()) {
    return this.calculateTotal(items)
  }
  return this.calculateTotalLegacy(items)
}
```

Continuous Refactoring (1/2)

Quality Gates Automatizados:

Pre-Commit:

- ESLint para code smells
- Unit tests pasan
- No nuevos magic numbers

PR Review:

- SonarQube quality gate
 - Coverage threshold
 - Performance regression tests
-

Continuous Refactoring (2/2)

Post-Deploy:

- E2E tests en producción
 - Error rate tracking
-

Refactoring Culture

Principles:

- Refactoring es higiene, no opcional

- Boy Scout Rule religiosamente
- Red-Green-Refactor cycle sagrado
- Tests nunca se rompen

Practices:

- Code reviews enfocados en diseño
- Pair programming para refactors complejos
- Knowledge sharing sobre patrones

Ejercicio 1: Extract Constant con Tests

Prompt:

Actúa como un desarrollador practicando Safe Refactoring con TDD.

CONTEXT0: Safe Refactoring significa reestructurar código sin cambiar comportamiento externo. Martin Fowler's principio: preservar funcionalidad idéntica. Tests actúan como "Safety Net" (red de seguridad): si tests pasan antes Y después del refactor, comportamiento está preservado. Extract Const es refactoring de nivel 1: bajo riesgo, alto impacto.

TAREA: Refactoriza magic numbers a named constants usando Test Safety Net.

PREPARACIÓN:

- Branch: refactor/01-smells (tiene magic numbers intencionales)
- Tests existentes DEBEN pasar antes de empezar

REFACTORING STEPS:

1. Ejecutar tests: `pnpm test` (deben pasar ✅)
2. Identificar magic numbers: 0.1 (bulk discount), 5 (threshold)
3. Extraer a `businessRules.bulkDiscount.percentage`
4. Reemplazar todas las ocurrencias del magic number
5. Ejecutar tests nuevamente: `pnpm test` (deben seguir pasando ✅)

UBICACIONES A REFACTORIZAR:

- `src/features/shopping-cart/components/CartSummary.tsx`
- Buscar: 0.1, 5, 100, 0.15 (magic numbers)
- Centralizar en: `src/shared/constants/businessRules.ts`

SAFETY NET:

- Tests que validan cálculos de descuentos
- Si tests fallan después del refactor → revertir cambios

- Refactoring exitoso = tests verdes antes Y después

VALIDACIÓN: ejecuta `pnpm test` → todos los tests deben PASAR ✅

Aprende: Extract constant mejora legibilidad sin romper tests

Ejercicio 2: Extract Function con TDD

Prompt:

Actúa como un desarrollador aplicando TDD para Safe Refactoring.

CONTEXT0: TDD para refactoring sigue Red-Green-Refactor: 1) Test que falla (RED), 2) Código mínimo para pasar (GREEN), 3) Mejorar diseño (REFACTOR). Extract Function es patrón fundamental: tomar código repetido o complejo y extraerlo a función reusable. Tests garantizan que función extraída tiene comportamiento idéntico al código original.

TAREA: Extrae lógica de cálculo de impuestos a función `calculateTax()` usando

TDD CYCLE:

1. 🚫 RED: Escribir `test` PRIMERO (función no existe aún)
 - Test: `calculateTax(100, 10)` debe retornar `10`
 - Test FALLA: `"calculateTax is not defined"`
2. 🟢 GREEN: Implementar función mínima
 - Función: `calculateTax(amount, rate)` retorna `amount * (rate / 100)`
 - Test PASA ✅
3. ♻️ REFACTOR: Mejorar si es necesario
 - Agregar validaciones, tipos TypeScript
 - Tests siguen pasando ✅

FUNCIÓN SPECS:

- Nombre: `calculateTax`
- Parámetros: `amount: number, rate: number`
- Retorno: `number` (tax amount)
- Fórmula: `amount * (rate / 100)`

TEST SPECS:

- Framework: Vitest
- Estructura: AAA pattern
- Test **case**: `calculateTax(100, 10) → 10`

ARCHIVOS:

- src/shared/utils/calculateTax.ts (función)
- src/shared/utils/calculateTax.test.ts (`test`)

VALIDACIÓN: ejecuta `pnpm test` → `test` debe PASAR 

Aprende: TDD asegura que refactoring preserve comportamiento

Puntos Clave

1. **Test-Driven:** Tests como red de seguridad inquebrantable
2. **Incremental** (incremental): Pequeños pasos seguros
3. **Behavior-Preserving** (preservar comportamiento): Funcionalidad idéntica
4. **Quality-Improving** (mejorar calidad): Mejor diseño interno
5. **Continuous** (continuo): Parte integral del desarrollo
6. **ROI** (Return on Investment - retorno sobre la inversión): Se paga rápidamente